

Manutenção Preditiva Industrial com Som

Bombas industriais · detecção de anomalia por áudio · Indústria 4.0



4.205 cliques · **96** features · **97,98%** de acurácia

PILAR 1

Dados & Pipeline

PILAR 2

Machine Learning

PILAR 3

Cloud & Dashboard

O Problema de Negócio



Esta bomba está com anomalia?

Devo parar para manutenção agora?

Dezenas de bombas operam 24/7. Parada não programada custa caro: produção parada + reparo emergencial. Manutenção desnecessária também tem custo.



Cenário

Fábrica real, bombas 24/7



Decisor

Engenheiro de manutenção



Risco

Parada × manutenção em excesso



Menos paradas não programadas



Menos manutenções desnecessárias



Economia em reparos emergenciais

Os Dados — Dataset MIMII



Bombas industriais reais

- 4 modelos de bombas
- Audios de 10 segundos
- Ruído real de fábrica

Zenodo · CC BY-SA 4.0 · arXiv:1909.09347

4.205

clipes de áudio

456

com anomalia

3.749

funcionamento normal

Distribuição das condições

normal · 89,1%

anomalia · 10,9%



Pipeline ELT — 8 Etapas



Extração de Features do Áudio

Áudio .wav



librosa (extração)



96 features numéricas



MFCC · 80 features

Timbre do som industrial — média e desvio de 40 coeficientes



Espectrais · 11 features

Centroid, bandwidth, rolloff e contraste em 7 bandas



Energia & Ritmo · 2 features

Zero-Crossing Rate e RMS — energia e potência do sinal



Metadados estruturados · 7

tipo · modelo · condição · duração · canais



Dataset para ML · 103 colunas

96 features + 7 metadados × 4.205 linhas

Airflow e dbt — Orquestração



Apache Airflow

- DAG com 8 tasks sequenciais
- Agenda, executa e monitora o ELT
- Reexecução e retries em produção



dbt — transformação

- 5 modelos: views e tables
- 14 testes de qualidade
- not_null · unique · accepted_values

Staging

view



Dim / Fact

table



Marts

table



ML

features

O Modelo — Rede Neural (MLP)

Arquitetura



MLP binário — $P(\text{anomalia})$ em $[0,1]$ · threshold 0,5



O que o modelo decide

- Entrada: 96 features do áudio
- 2 camadas ocultas (64 → 32)
- Saída: $P(\text{anomalia}) \in [0,1]$
- Threshold 0,5 → decisão binária

Implementado 2x: NumPy (hard-code) e sklearn.

Hard-Code vs Sklearn



Hard-code — do zero (NumPy)

- Forward: ReLU + Sigmoid
- Backprop: gradientes analíticos
- SGD com momento (lr 0,01 · 0,9)
- Binary cross-entropy · 300 épocas
- Sem TensorFlow, PyTorch ou autograd



Sklearn — MLPClassifier

- Mesma arquitetura (64, 32)
- Adam · max_iter 500
- Regularização L2 (alpha 0,0001)
- Código C otimizado (~6,7× mais rápido)

```
h = ReLU(W1·x + b1) → p = sigmoid(W2·h + b2)
L = -[ y·log(p) + (1-y)·log(1-p) ]
```

Resultados e Métricas

97,98%

Acurácia · MLP sklearn

0,899

F1-Score · anomalia

83,5%

Recall · anomalias detectadas

Comparação entre modelos

Modelo	Acurácia	F1
Baseline majoritária	89,18%	0,000
Regressão logística	96,79%	0,832
MLP hard-code	97,86%	0,894
MLP sklearn	97,98%	0,899

Matriz de confusão · MLP sklearn

	Predito normal	Predito anomalia
Real normal	748	2
Real anomalia	15	76

FP 2 (falsos alarmes) · FN 15 (anomalias sutis — mascaradas pelo ruído de fábrica)

Arquitetura 100% AWS



Dev ↔ Prod e Infraestrutura



Dev ↔ Prod — mesmo código

- Troca por 4 variáveis de ambiente
- S3: MinIO ↔ AWS (endpoint URL)
- Banco: PostgreSQL ↔ Snowflake
- Zero mudanças de código



Infraestrutura como código

- AWS CloudFormation (template YAML)
- S3 · EC2 · rede · segurança
- Registro de custos no relatório



Ambiente 1 — Local (Docker)

MinIO + PostgreSQL + Airflow + Metabase



Ambiente 2 — Produção (AWS)

S3 + EC2 + Snowflake + SageMaker

Dashboard e Decisão Final



Metabase — visão gerencial

- KPIs: amostras, taxa de anomalia, duração média
- Filtro por modelo de bomba
- Anomalias por modelo de máquina
- Matriz de confusão e previsões com erro

Decisão de manutenção

$P(\text{anomalia}) > 0,5$

-  Normal — continue operando
-  Anomalia — agende manutenção

O engenheiro recebe o alerta no dashboard e decide com base em dados.

Áudio + dados + ML + nuvem apoiando a decisão do engenheiro, em tempo real.

Obrigado!